

Doc. no.: P0608R0

Date: 2017-03-02

Audience: LEWG, LWG

Reply-to: Zhihao Yuan <zy at miator dot net>

A sane variant converting constructor (LEWG 227)

Rationale

See [LEWG 227](#).

Wording

This wording is relative to N4640.

Modify 20.7.3.1 [variant.ctor]/12 as indicated:

```
template <class T> constexpr variant(T&& t) noexcept(see below );
```

In the following sections, let *is_non_narrowing_convertible_v* be an exposition only trait defined as:

```
template <typename From, typename To>
constexpr bool is_non_narrowing_convertible_v = see below;
```

is_non_narrowing_convertible_v<From, To> is true if *is_convertible_v*<From, To> is true and converting a non-constant object of From to To does not require a narrowing conversion (8.6.4) or a boolean conversion (4.14), otherwise false.

Let τ_j be a type that is determined as follows: build an imaginary function $FUN(\tau_i)$ for each alternative type τ_i , where $FUN(\tau_i)$ shall not participate in overload resolution unless *is_non_narrowing_convertible_v*< τ , τ_i > is true. The overload $FUN(\tau_i)$ selected by overload resolution for the expression $FUN(std::forward<\tau>(t))$ defines the alternative τ_j which is the type of the contained value after construction.

[...]

Modify 20.7.3.3 [variant.assign]/8 as indicated:

```
template <class T> variant& operator=(T&& t) noexcept(see below );
```

Let τ_j be a type that is determined as follows: build an imaginary function $FUN(\tau_i)$ for each alternative type τ_i , where $FUN(\tau_i)$ shall not participate in overload resolution unless *is_non_narrowing_convertible_v*< τ , τ_i > is true. The overload $FUN(\tau_i)$ selected by overload resolution for the expression $FUN(std::forward<\tau>(t))$ defines the alternative τ_j which is the type of the contained value after assignment.

[...]